DAX Logical Functions

The deck combines concise definitions, syntax boxes, illustrative examples, and plentiful imagery to help you learn and apply logical functions in production measures and calculated columns.

Introduction: What and Why for Logical Functions in DAX

Logical functionsevaluateBooleanconditions andcontrolflow inside DAX expressions. They are essential for classification, conditional aggregation, error handling, and building readable measures. Logical evaluation influences filter context, row context, and the order of computation4key concepts for accurate analytics in Power BI models.

- Role: Branching and condition evaluation in measures and calculated columns.
- Impact: Affects performance and filter propagation (CALCULATE interplay).
- Audience: Analysts designing measures, modelers optimizing performance.

Use logical functions to: implement business rules, perform conditional formatting, sanitize missing data, and combine multiple boolean checks. This section prepares you to read the reference pages that follow.

Overview Table: Logical Functions at a Glance

Quick reference listing of DAX logical functions included in this guide.

Function Name	Purpose	Return Type	Example Use Case
AND()	Logical conjunction of two expressions	Boolean	Eligibility checks Multi-condition flags
OR()	Logical disjunction of two expressions	Boolean	Inverse conditions Simple branching
NOT()	Negates a Boolean	Boolean	Prevent errors in nested logic Multi-condition mapping
IF()	Conditional selection (true/false branches)	Any	Default flags Null substitution Category matching
IF.EAGER()	Early evaluation IF (short-circuits)	Any	Inverse conjunction
SWITCH()	Multi-branch selector	Any	
TRUE() / FALSE()	Boolean constants	Boolean	
COALESCE()	First non-blank value	Value type of args	
IN()	Value membership in set	Boolean	
NAND()	Not AND (if available)	Boolean	

Colors used for emphasis in examples: primary #282824, accents #E8E4DD and #DED8CD.

AND(), OR(), NOT() 4 Basic Boolean Operators

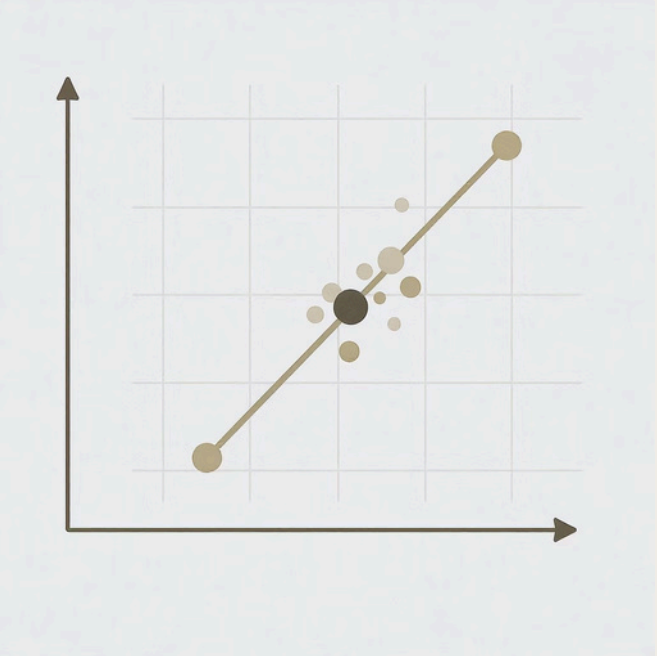
AND()

Definition: Returns TRUE if both expressions evaluate to TRUE. Syntax: AND(,)

Parameters: Both arguments must return Boolean-compatible values. Return value: Boolean.
Example:

```
IsHighValue = AND([Sales] > 10000, [ProfitMargin] >= 0.2)
```

Use case: Mark rows meeting combined thresholds for segmentation.



OR()

Definition: Returns TRUE if at least one expression is TRUE. Syntax: OR(,)

Example:

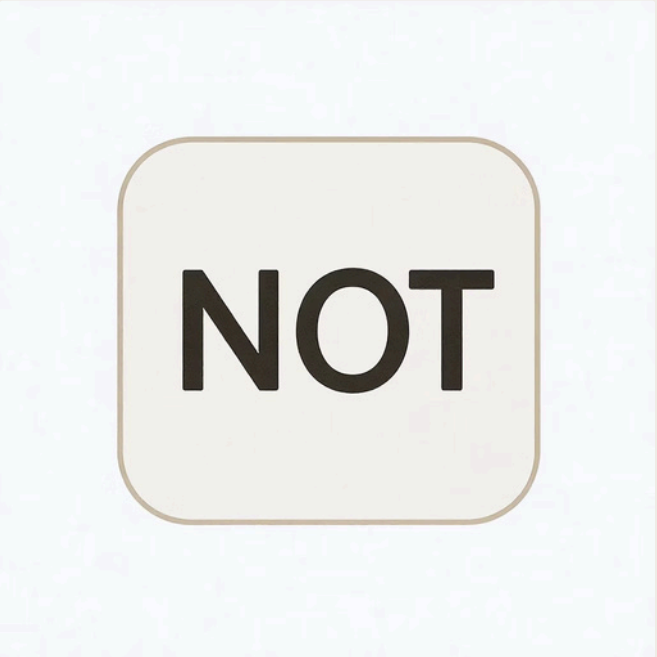
```
IsPriority = OR([CustomerTier] = "Platinum", [BacklogDays] > 30)
```

NOT()

Definition: Returns the logical negation. Syntax: NOT()

Example:

```
IsNotActive = NOT([IsActive])
```



IF() and IF.EAGER() 4 Conditional Evaluation & Short-circuiting

IFisthefundamental conditionalfunctionin DAX.IF.EAGERenforcesearly/short-circuitevaluation to avoid evaluating branches that would cause errors or heavy computation.

1

IF()

Syntax: IF(, , [])

Example:

```
ProfitFlag = IF([Profit] < 0, "Loss", "OK")
```

Notes: The false branch is optional4returns BLANK if omitted.

2

IF.EAGER()

Syntax: IF.EAGER(, , [])

Example (avoid divide-by-zero):

```
SafeRatio = IF.EAGER([Denominator] <> 0, [Numerator] / [Denominator], BLANK())
```

Use case: Prevent unnecessary evaluation of heavy expressions or those that cause errors.

Best practice: Use IF.EAGER when nested branches contain calculations that may error or take significant time; otherwise prefer IF for readability.

SWITCH(), TRUE(), FALSE() 4 Multi-branch Selection & Constants

SWITCH is a readable alternative to deeply nested IFs when selecting between multiple discrete outcomes.

SWITCH()

Syntax (value form): SWITCH(, , , [,], ..., [])

Example:

```
RegionGroup = SWITCH([RegionCode], "NE", "Northeast", "SE", "Southeast", "Other")
```

TRUE()/FALSE()

Definition: Return Boolean constants. Useful inside SWITCH(TRUE(), ...) pattern to evaluate arbitrary conditions in sequence.

Example (SWITCH with conditions):

```
RiskLevel = SWITCH(TRUE(), [Score] >= 80, "Low", [Score] >= 50, "Medium", "High")
```

COALESCE(), IN(), and NAND() 4 Null Handling & Membership

COALESCE()

Definition:Returns the first non-blank value from its arguments. Syntax: COALESCE(, , ...)

Example:

```
DisplayName = COALESCE([FullName], [PreferredName], "Unknown")
```

Use case: Replace BLANKs, provide fallbacks for missing values, simplify nested IFs.

IN()

Definition: Tests member existence within a list. Syntax example: [Category] IN {"A", "B", "C"}

Example:

```
IsPromoCategory = IF([ProductCategory] IN {"Promo","Seasonal"}, TRUE(), FALSE())
```

NAND(): Not an official core DAX function in many references; if available via custom logic, it9s equivalent to NOT(AND(...)). Use NOT(AND(...)) for clarity and compatibility.

Comparative Table: IF() vs SWITCH() vs COALESCE()

Function	Behavior	EvaluationType	Whento use
IF()	Binary branching (true/false)	Evaluates condition, then one branch	Simple checks or two-way branches
SWITCH()	Multi-case selection	Evaluates expression or sequence (SWITCH TRUE pattern)	Multiple discrete outputs, clearer than nested IFs
COALESCE()	Returns first non-blank	Evaluates arguments left to right	Null replacement, fallback values

The table highlights evaluation intent: SWITCH improves readability for many mutually exclusive branches; IF.EAGER helps avoid unnecessary evaluation that could cause errors.

Real-World Use Cases & Flow Diagram

Three practical scenarios and a logical flow diagram illustrating how functions combine in measures.

Conditional Formatting

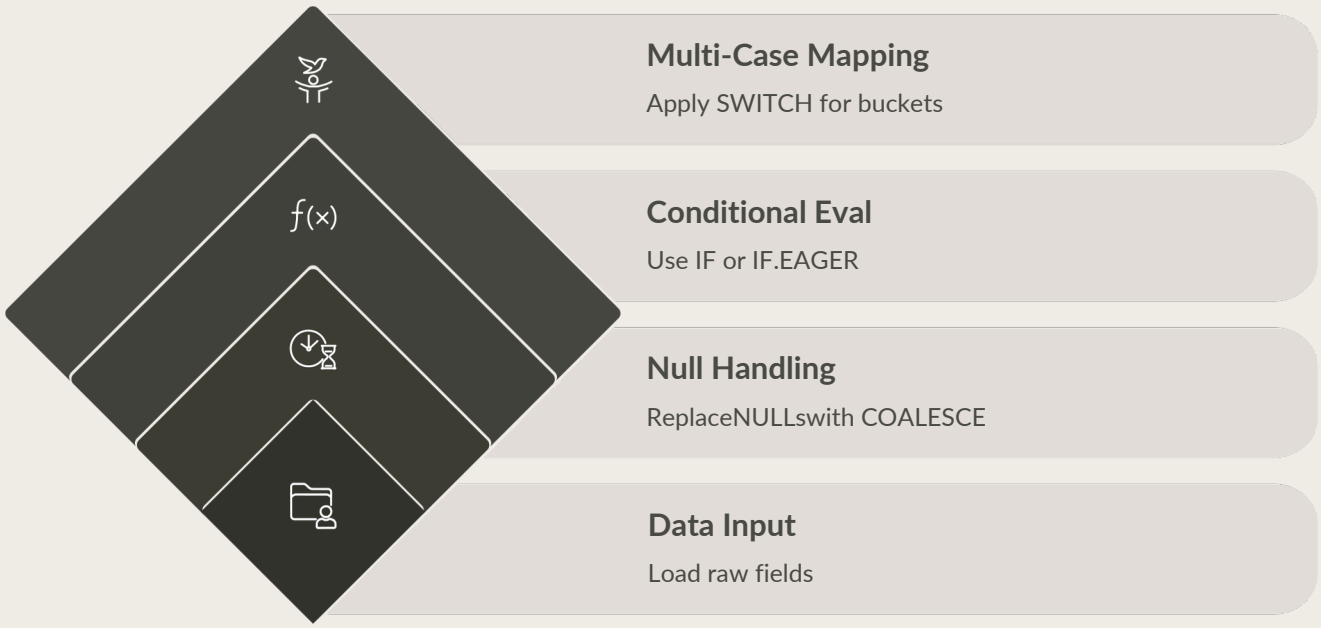
Use SWITCH(TRUE(), ...) or nested IFs to compute color buckets used in conditional formatting measures.

Missing Data Handling

COALESCE replaces BLANKs for reporting and prevents empty labels in visuals.

Complex Segment Logic

Combine AND, OR, and IN to build precise segment flags used across measures and slicers.



Tips, Best Practices, and Key Takeaways

Performance

Avoid heavy calculations inside repeatedly evaluated branches. Use variables (VAR) to reuse results and IF.EAGER to short-circuit dangerous expressions.

Readability

PreferSWITCH for multiple discrete outcomes. Keep nested logic minimal4document intent with comments in editor where possible.

Filter Context

Rememberlogical function results interact with filter context. Use CALCULATE and KEEPFILTERS carefully when conditions depend on context.

Robustness

UseCOALESCE to handle BLANKs and IF.EAGER to avoid errors from division or lookup failures. Prefer explicit checks over relying on implicit conversion.

Summary: Logical functions are foundational for DAX expression control flow. Use the right tool4IF, SWITCH, COALESCE, IN, AND/OR/NOT4and apply short-circuiting and variable reuse to create maintainable, performant measures. For further reading, consult Microsoft Learn DAX documentation (2025).